Scoping of Proposed Dash Projects (July 2026, v4.4)

Hilawe Semunegus, with the originating ideas, roadblock framing, and wallet-policy design by Joel Valenzuela (TheDesertLynx)

This document sizes up candidate Dash projects aimed at four roadblocks, pooled masternodes, speed of capital movement to and from Platform, DashPay usability, and privacy. Versions 1 through 3 scoped seven separate projects, P1 through P7. Version 4 restructures them into four, because the ground moved. The Orchard shielded pool ships with Platform v4.0, which has locked in on the network with activation scheduled on or around July 12, 2026 (per the Dash roadmap and the public Dash Core Group development updates of June 25 and July 9, 2026), and several of the original projects turn out to be halves of the same thing once the pool is real. The old numbering is kept in parentheses so earlier discussion stays traceable.

Each section opens with a Plain words explainer for the general reader and then gets into the technical detail. The revision history and credits are at the end.

Summary

#	Project	Was	Layer	Size	Verdict
D1	Instant, private credit withdrawals	P1 + P7	Core + Platform	D0 audit done; Path A (policy, S-M) confirmed	Do first; audit confirms payout invariant holds, spec request filed upstream
D2	Batch-rotation wallet policy	P4 re-designed; P5 dropped	Mobile	M prototype + measure-ment gate	Contingency; D1 first, build only if per-payment withdrawals prove capped, costly, or slow
D3	Private payments to any username	P3 + P6	Platform + Mobile, Core research annex	M (payment codes); L (static-address annex)	Design now alongside the contact-requests v2 work, build when Platform stabilizes
D4	Coinbase maturity relaxation under ChainLocks	P2	Core	S code, L process	Fold into a future hard fork, not standalone

D1. Instant, private credit withdrawals (was P1 and P7)

Plain words. Dash Platform keeps its own balance system, called credits, and with the v4.0 upgrade those balances can be shielded, hidden amounts, hidden sender and recipient, with view keys for anyone who needs to show their books. Depositing DASH into Platform melts it into a shared pool, and withdrawing mints a brand-new transaction with no inputs, so on the main chain

there is no thread from the withdrawal back to the deposit. Private money in, private money held, and a clean exit out. What is missing is speed. The receiving exchange or merchant still waits about 2.5 minutes for a mined, ChainLocked block before crediting a withdrawal, and that wait is now the last protocol-side gap in an instant, private cash-out.

The fix may be cheap, because the wait exists for a narrow reason. A withdrawal is already signed by a quorum of masternodes before it appears, so it is already authorized, and with no inputs there are no coins to double-spend. The one loose end is expiry. A withdrawal that sits unmined for 48 blocks becomes invalid, and the rules today let Platform retry it without promising that the retry pays the same place the same amount, or that a retry happens at all. Write that promise into the rules and wallets can credit a withdrawal the moment it appears in the mempool.

Details. A withdrawal from Platform arrives on the Core chain as an asset unlock transaction (DIP-27, special transaction type 9). These transactions have no inputs. Each one carries a withdrawal index that must be unique, an explicit miner fee, and a signature from a Platform validator quorum (LLMQ_100_67 on mainnet). The rules refuse them once the chain height exceeds their signing height by 48 blocks, they are not eligible for InstantSend, and finality comes only from being mined into a ChainLocked block.

Because the transaction is already quorum-signed when it arrives, authorization is not the gap. The gap is expiry. DIP-27 says an expired withdrawal "can be retried", but it does not bind the retried transaction to the original payout script and amount, does not require a retry to happen, and does not define an abandoned state a receiver could detect.

The project started with a short audit, phase D0, published as d0-audit-retry-payout.md. It read both repositories at pinned commits and answered the question, does a withdrawal index always bind to exactly one payout across retries? It does. On expiry and re-sign Platform keeps the same index, output script, amount, and fee, changing only the signing height and quorum, and Core lets an index settle at most once on the chain. So a valid quorum-signed unlock held in a receiver's own fully validating Core node will settle as exactly what it shows, and instant acceptance with deduplication by index is payout-safe (final settlement still keys to ChainLock). That confirms Path A below. The audit also surfaced two withdrawing-user edge cases for the spec to close, a permanently stuck withdrawal is debited with no refund, and the fee is fixed across retries so a withdrawal created with too low a fee can strand.

Timing makes D0 more valuable than it first appears. The June 2026 development update lists Asset Locks v2 in the active specification pipeline, so the withdrawal rules are being revised right now. The requirement has already been drafted as a one-page upstream request, asset-locks-v2-retry-payout.md, payout bound to the index at request acceptance, fee adjustable but never taken from the payout, retry-until-mined as the liveness rule, and receiver deduplication by withdrawal index. The liveness form matters, an accepted withdrawal must be a promise to pay, not a promise to try, because any abandonment path demotes pre-mining credit from final to provisional and cautious receivers would keep waiting for blocks. Landing this in the v2 draft is the cheapest moment such a guarantee will ever have.

- Path A, the guarantee holds or lands in Asset Locks v2 in its strong form. Instant acceptance is then pure receiving policy. Wallets and exchanges credit a valid quorum-signed unlock that their own node holds in its mempool, and published integrator guidance spreads the practice. No Core consensus change and no new network messages. Payout-safety is the property that holds here, the receiver knows what will settle. The remaining risks, non-inclusion and

eviction, are addressed by retry cycling, but note the audit's finding that retry re-signs at a fixed fee, so the fixed-fee stranding case closes only if fee-bump-on-retry is also adopted. Size, small to medium.

- Path B, the guarantee cannot be made, or attested finality is wanted for light clients and for chaining child transactions. Then a new quorum lock message binds the index, the txid, and the payout together. That design has real work in it. The lock must expire with the transaction it covers, unlike regular InstantSend locks, which never expire. It needs defined behavior when a conflicting unlock is mined into a ChainLocked block. Child transactions spending a locked unlock's outputs need an explicit eligibility rule change before they can receive ordinary InstantSend locks. Quorum signing load, denial-of-service limits, persistence, the P2P and RPC surface, and reorg rules for the new conflict domain all need design, and the delivery cost includes tests, deployment, integrator rollout, and the DIP process. Size, large.

One boundary caveat keeps this honest. A shielded pool is only as private as its entrances and exits. Deposits and withdrawals carry visible amounts and timing on the Core chain, and matching money going in against money coming out is how observers reconnect what the pool hid. Speed (this project) and boundary hygiene (D2) are therefore two halves of the same outcome, an exit that is both instant and genuinely private.

D2. Batch-rotation wallet policy (was P4, redesigned; absorbs the goal of dropped P5)

Plain words. Keep savings shielded, and keep a small transparent pot for everyday spending. The pot has a floor and a ceiling, say 5 and 10 DASH. It starts by pulling roughly 10 from the shielded balance to a fresh address, and normal spending happens from it, change addresses, combined inputs, single-address invoices, all ordinary wallet behavior. When the pot drops below the floor, the wallet rotates. A fresh draw lands on a brand-new address as a new, sealed batch, and the old batch's leftovers go back to the shielded balance later, in random pieces, at random times. Coins from different batches are never spent together. To an observer, each batch is a disposable mini-wallet with no visible relation to the ones before or after it. Financial history gets chopped into short, unlinkable chapters.

This design, developed by Joel Valenzuela through several rounds of refinement, replaces the earlier no-merge proposal (P4), which bought weaker privacy at a higher price. Within a batch the wallet behaves completely normally, so invoices, merchants, and exchange deposits keep working, which is the thing the old proposal could not do. One caveat from its own designer changes the verdict, though. If credit withdrawals get an instant finality guarantee (project D1), most users may not need any of this, because they can pay straight from the shielded balance per purchase. The details below end with why the design is kept anyway, as a contingency rather than a build commitment.

Details. The policy in its current form:

- A transparent spending band with a floor and a ceiling, refilled from the shielded balance.
- Batch isolation. Coins from different refills are never co-spent in one transaction, so the common-input-ownership heuristic can cluster within a batch but never across batches.
- Randomized refill amounts, not round numbers, so refills blend with organic pool exits. At wide adoption the logic inverts and standardized amounts become the stronger crowd, so the parameter should be revisitable.

- The leftover-return is decoupled from the refill, sent later, in random-sized pieces, at random times, with several old batches coexisting, so exits cannot be paired with entries.
- The return is always slower than the refill, and only one rotation runs at a time. Liquidity never depends on the return queue, because a user who outspends the rotation falls back to paying directly from the shielded balance. The full refill, return, and queueing state machine, including how many dead batches may coexist and what happens when spending outruns rotation, is a required part of the specification, and the liquidity promise is only as good as those rules.
- Large payments skip the pot entirely and go straight from the shielded balance to the recipient, which also handles payments bigger than the current batch.
- All broadcasts route over Tor, since network origin would otherwise re-link everything the chain-level rules separated.
- Dust in dead batches gets swept back slowly, one batch at a time, never combined.

Six residual risks the prototype has to answer, identified across the design reviews:

- Trigger observability. An observer who has clustered a batch can watch its balance sink below the floor and then look for a fresh pool exit. Randomized delays widen the window, but refilling proactively on a randomized schedule, so no visible dip-then-refill sequence exists, closes it. The spec should adopt schedule-driven refills.
- Counterparty reuse. Two batches paying the same merchant address are re-linked instantly at the application layer. The wallet should warn on cross-batch counterparty reuse or route repeat counterparties directly from the shielded balance.
- Wallet fingerprints. Batches produced by the same software can be clustered statistically by transaction shape, fee policy, and output ordering, so the wallet's shapes need to match the dominant wallet's.
- Light-client lookup. Mobile wallets tell the servers they query which addresses they watch, which would re-link batches server-side regardless of Tor broadcast. Private address lookup for light clients is required, and it is the same infrastructure problem as D3's scanning annex, so it should be specced once for both.
- State recovery. Batch membership, pending return queues, and counterparty history are wallet metadata that a seed phrase cannot regenerate. A naive restore would either strand queued returns or silently co-spend across batches and undo the privacy model. The specification needs an encrypted metadata backup, and a safe degraded restore mode that treats all recovered coins as one sealed batch to be swept through the shielded balance rather than merged with new activity.
- Fee and mempool behavior. Refill, return, and dust-sweep transactions must clear under mempool minimum fees and fee spikes, and fee-driven timing can itself mark rotation traffic or stall the return queue. Fee policy is part of the privacy design, not only a cost line.

The whole design assumes a healthy, busy shielded pool, since the crowd is the privacy, and that assumption degrades honestly at low adoption, with few pool users, boundary events stand out no matter how well they are randomized, so early measurements should model small anonymity sets rather than assume mature ones. It also composes with, rather than replaces, the pool itself, batch rotation is the boundary hygiene that keeps entrances and exits from undoing what the pool hides. Where the shielded balance is not yet available to a given wallet, a mixed reserve can stand in with the same rotation mechanics.

The measurement gate before shipping, prototype behind a toggle with measured outputs. Boundary-correlation resistance under simulation at both small and realistic adoption levels (can a timing-and-amount correlator re-link batches). Fee overhead per rotation epoch. Stranded-dust rate. A restore drill, recover from seed plus metadata backup mid-rotation without batch merging or fund loss. Zero regressions on invoice and deposit flows. Ship or kill on those numbers.

Does D1 make this unnecessary? The designer's own assessment, delivered with the final policy, is that the whole scheme is redundant if credit withdrawals get an instant finality guarantee, because a wallet could then pay every recipient directly from the shielded balance, one fresh zero-input transaction per payment, no pot, no batches, no rotation. For sender clustering on the Core chain that is correct and better, each payment is a fresh zero-input transaction with no past and no common-input link to any other payment, while batch rotation still correlates everything inside a batch. The boundary caveat from D1 still applies, every exit publishes an amount and a timestamp, so many small payment-shaped exits can still leak through amount patterns, timing, recipient reuse, network origin, and Platform-side metadata, which is why Tor and recipient hygiene stay relevant in either design. Three considerations keep D2 as a contingency rather than a deletion. Throughput, DIP-27 limits how much can leave the credit pool over its 576-block safety window (withdrawal limits are on the spec's own list of reasons an unlock may fail to mine), and a network where every small purchase is a quorum-signed withdrawal presses on those limits and on the signing pipeline, so high-frequency small spending may still need a transparent pot. Cost, every payment becomes a Platform state transition plus a Core transaction, and fee floors price out the micropayments a pot amortizes. And insurance, if D1 stalls or lands in its weak form, pot-and-rotation is the fallback that works without it. The sequencing is therefore D1 first, then measure per-payment withdrawal fees, latency, and cap headroom, and build D2 only if those numbers say a pot is still needed.

Retired pieces, recorded for traceability. The per-payment no-merge policy (old P4) is withdrawn, its costs (double fees, split payments, invoice breakage) bought less privacy than batch rotation buys for less. The cross-chain consolidation route through Zcash (old P5) stays dropped, its full record is in the v2 and v3 revision notes, the short version is that cross-chain exchanges record swap addresses, amounts, and timing in clear on their own ledgers, so the round trip publishes what it was meant to hide, and with a native shielded pool arriving there is no reason to rent another chain's.

D3. Private payments to any username (was P3 and P6)

Plain words. DashPay usernames make paying people humane, but today a username can only be paid after both sides accept a contact request, because that handshake is where wallets exchange the keys that generate fresh addresses. Paying a stranger's username needs the recipient to publish something in advance, and what gets published decides the privacy. Publish a raw extended public key and anyone can watch every payment the account ever receives. The workable options publish less.

Details. Usernames live in the Dash Platform Name Service, and payment derivation lives in the DashPay contract (DIP-15), where mutually accepted contact requests exchange encrypted extended public keys. Four design options, in ascending privacy:

- A static address in the name record. The name service optionally resolves a username to one fixed address. Zero privacy, every payment lands on the same visible address, but it is

the simplest possible tip jar and costs nearly nothing to add. Useful as an explicit opt-in for public figures who want tips, never as a default.

- A payment code in the profile (BIP47-style, buildable with today's primitives). The sender derives a one-off P2PKH address from a published payment code, and posts a state transition as the notification telling the recipient's wallet which derivation to watch. Platform credit fees rate-limit notification spam. Three design questions need clearing before this is more than a sketch. Key separation, whether payment derivation uses a dedicated key rather than reusing Platform identity keys, which is a protocol choice with its own security analysis, not an implementation detail. Recovery, how a restored wallet finds its notification history. And graph visibility, the notification documents sit on Platform, and even with encrypted payloads the sender-to-recipient edge may be visible to observers, so the document and query model need a privacy review. With shielded state now shipping in Platform v4.0, a shielded variant of the notification is the natural upgrade path.
- A pay-to-decrypt one-time address through a revised contact-request flow. The proposer reports community work toward a second version of DashPay contact requests along these lines, which no public artifact yet confirms, so the claim is recorded here as unverified. The first task of the design document is to find that work if it exists and sync with it rather than fork, and the option stands on its own merits either way. Whatever ships as contact requests v2 should carry both single-use addresses and the friending flow.
- A silent-payment-style identifier (the research annex, old P3). One published identifier, no notification at all, every payment lands on a fresh address derived by the sender, and Platform serves purely as a name-to-identifier directory, so payments remain valid even when Platform is unavailable. The Bitcoin spec for this, BIP352, cannot be copied onto Dash because it assumes Taproot, which Dash does not have. A Dash-specific derivation on ECDSA and P2PKH is feasible, the underlying trick predates Taproot, and a Diffie-Hellman-tweaked P2PKH output is byte-identical to any other P2PKH output. The costs are a Dash-specific spec with its own security review, no reuse of upstream tooling, and the scanning burden, full nodes scan every transaction for their payments, and mobile clients need index servers publishing per-transaction hints, the same private-lookup infrastructure D2 needs. Research-gated, a derivation spec plus a scan-cost benchmark before any build commitment.

A priority note the shielded pool forces. With private balances and private transfers arriving natively on Platform, the case for heavy investment in L1 address privacy weakens, much of what silent payments bought is now bought by holding shielded credits and paying through Platform. Not all of it. Silent payments remain the only option with no notification of any kind and the only one that keeps Platform entirely out of the payment-critical path, and L1-native payments will exist for a long time. The annex is retained for those properties, but its sequencing moves behind the payment-code and revised contact-request options, which serve the username use case sooner and cheaper.

D4. Coinbase maturity relaxation under ChainLocks (was P2)

Plain words. When a miner finds a block, the reward inside it cannot be spent for 100 blocks, about four hours. The rule is inherited from Bitcoin and exists for one reason. If the chain reorganizes and the block gets orphaned, the reward inside it never existed, and anyone paid with it downstream would be holding air. Dash largely closed that door with ChainLocks, where a masternode quorum signs off on each block within seconds and makes reorganizations effectively

impossible. Once a block is ChainLocked, the risk the 100-block rule guards against is gone, so the wait is mostly a leftover.

Details. Removing a leftover sounds trivial, and the code is. The catch is that the maturity rule is consensus, every node enforces it, so relaxing it means a hard fork, and a hard fork is an expensive event no matter how small the code change. There is also a real design wrinkle. ChainLock signatures are not stored inside blocks, so a rule that says "spendable once ChainLocked" needs an answer for how a node syncing from scratch, years later, verifies that history. The sensible move is to solve that wrinkle on paper and attach the change to the next hard fork that ships for other reasons. Faster access to rewards helps miners, the masternodes paid in every block, and the treasury recipients paid in superblock coinbases on their own cadence.

Recommended sequence

1. D1 now. Send the Asset Locks v2 retry-payout request upstream while the spec is in flight, and run the D0 audit, one to two weeks, choosing Path A or Path B on its answer.
2. D2 held as a contingency. Finish the spec on paper (schedule-driven refills, counterparty-reuse handling, fingerprint discipline, private lookup, state recovery), but build only if the post-D1 measurements, per-payment withdrawal fees, latency, and cap headroom, say a transparent pot is still needed.
3. D3 as a design document now, synced with the contact-requests v2 work, payment-code and pay-to-decrypt options first, the silent-payment annex research-gated behind its derivation spec and scan-cost benchmark.
4. D4 folded into the next hard fork once its initial-sync question is answered.

Revision history

- v2 incorporated independent adversarial reviews by multiple AI models, plus primary-source checks against DIP-27, BIP352, and live decentralized-exchange pool data (2026-07-08). The material changes, P1 restructured around the D0 audit of the retry-payout invariant, P3 re-scoped for the missing Taproot dependency, P4 restricted to contact payments, P2's beneficiary claim corrected (masternodes every block, treasury through superblock coinbases), P5's evidence corrected in both directions, and P6's extended-public-key risk statement corrected.
- v2.1 corrected P5 again after a further review catch. Shielded ZEC swap support has shipped on Maya Protocol, so the claim that shielded notes cannot work in these swap architectures was withdrawn, and the drop argument now rests on the exchange-chain metadata leak and the economics.
- v2.2 added two design observations from a comparison with Quai's Qi ledger. P4 gained a denominated-payments hypothesis, and P7 gained the credit-pool consolidation-valve framing.
- v3 and v3.1 rewrote the document in plain language after community feedback, adding the Plain words explainers. The technical content was unchanged from v2.2.
- v3.2 through v3.4 corrected P7's status as the Orchard shielded pool integration became public and then locked in, activation on or around July 12, 2026, and pointed P1's audit at the in-flight Asset Locks v2 specification.
- v4 restructures seven projects into four after co-design with Joel Valenzuela. P1 and P7 merge into D1 (the shielded half now ships upstream, so the boundary is the whole project). P4 is replaced by the batch-rotation wallet policy as D2, with the old no-merge policy withdrawn

and P5's record folded in. P3 and P6 merge into D3 with four design options and the silent-payment work re-sequenced as a research annex. P2 continues unchanged as D4. The companion one-page spec request for D1 is published alongside this document.

- v4.4 adjudicates a second full three-model round on the whole document set. Two of the three independent reviews confirmed no source path to a divergent payout, so the Path A conclusion stands, and both sharpened the same theme, the guarantee is a Platform-layer implementation property, not yet a Core consensus invariant, and retry cycling at a fixed fee is not the same as guaranteed mining. Fixes, a corrected withdrawal-limit constant in the audit (2000 DASH live, 4000 only under the inactive V24), the Platform-vs-consensus distinction added to the audit and the one-pager, "closed by retry" softened, mempool guidance tied to the receiver's own validating node, and the one-pager's first ask recast to reflect the audit's answer. The third review reproduced its prior round's findings against superseded text and was set aside after verification.
- v4.3 records the D0 audit result. The retry-payout invariant already holds in the Platform implementation, so D1 is Path A, instant acceptance is a receiving-policy plus spec write-down, no protocol change. The audit is published as a companion document and surfaced two withdrawing-user edge cases (no refund on a stuck withdrawal, fixed fee stranding) for the spec to close.
- v4.2 demotes D2 from build-after-activation to contingency, following the designer's own assessment that instant withdrawal finality (D1) lets wallets pay per purchase directly from the shielded balance. The adjudication keeps D2 for three reasons, DIP-27 withdrawal limits, per-payment fee floors, and insurance against D1 stalling or landing in its weak form, with post-D1 measurements deciding whether it is ever built.
- v4.1 adjudicates a full three-model adversarial round on v4 and the companion request. The convergent findings, all fixed, the liveness rule is strengthened to retry-until-mined because any abandonment path demotes instant credit from final to provisional, D2 gains two residual risks (state recovery across seed restore, fee and mempool behavior) plus an explicit liquidity state-machine requirement and a restore drill in the measurement gate, the revised contact-request claim in D3 is marked unverified pending a public artifact, the payment-code option's key-separation and recovery questions are stated, and the companion request's cross-references now use the v4 numbering. One review disputed the shielded-pool activation itself, that finding was rejected against the Dash roadmap, the June 25 and July 9 development updates, and the two concurring reviews.

Credits

The originating proposals, the four-roadblock framing, and the batch-rotation wallet policy design are due to Joel Valenzuela (TheDesertLynx). The scoping, the verdicts, and any errors are the maintainer's.

The claims in this document were adversarially reviewed by multiple independent AI models over several rounds, and every disputed claim was adjudicated against primary sources, the dashpay/dips and bitcoin/bips repositories, Dash Core source, and live decentralized-exchange pool data.